

Use Case Descriptions

Networking Team

Use case: Make a move

Iteration: 1

Primary Actor: Player

Goal in context: To ensure server-client interaction by efficiently validating player move and then transmitting them to the game server.

Preconditions:

1. Player logs into game platform.
2. The player is logged into the game.
3. The player is in an active game session.
4. It is the player's turn.

Trigger: Player makes a move on the game board.

Scenario:

1. Player makes a move.
2. The move is validated and sent to the server.

Post conditions: Move is validated and updated on the server.

Exceptions: The player has a poor network connection, causing delays or failure in move synchronization.

Priority: High (To continue a game a move should be made by each player alternatively).

When available: 3rd iteration

Frequency of use: High (To continue a game a move should be made by each player alternatively).

Channel to actor: Internet connection, screen, keyboard, mouse.

Secondary actors: Game server

Channel to secondary actors: Internet connection (Wi-Fi, mobile data, or Ethernet).

Open issues: What happens if a player never makes a move?

Use case: Game chat

Iteration: 1

Primary Actor: Player

Goal in context: To allow players to communicate with each other in real-time.

Preconditions:

1. Player logs into game platform.

2. The player is logged into the game.
3. The player is in an active game session.

Trigger: Player sends/receives message form/in the chat box.

Scenario:

1. 1.Player writes a message in the chat box and sends it.
2. The message is sent to server and server forwards message to recipient.
3. Opponent players receive a message in the chat box.

Post conditions: Message appears in the chat box of recipient player.

Exceptions: Message do not gets delivered due to network issues.

Priority: Low (A game can continue without sending or receiving messages).

When available: 3rd iteration

Frequency of use: Low(A game can continue without sending or receiving messages).

Channel to actor: Internet connection, screen, keyboard, mouse.

Secondary actors: Game server

Channel to secondary actors: Internet connection (Wi-Fi, mobile data, or Ethernet).

Open issues: Are players allowed to use offensive words in the chat?

Use case: End turn

Iteration: 1

Primary Actor: Game server

Goal in context: To end one player's turn and initiate the other player's turn. So that turns alternate between the two players after each move.

Preconditions:

1. Player logs into game platform.
2. The player is logged into the game.
3. The player is in an active game session.
4. It is the player's turn.

Trigger: Player makes a move

Scenario:

1. Player's move is validated and sent to the server.
2. Server receives move information and player turn ends.

Post conditions: The first player's turn ends, and the second player's turn begins.

Exceptions: The player has a poor network connection, causing delays or failure in move synchronization.

Priority: High (To continue a game turn should alternate between two players).

When available: 3rd iteration

Frequency of use: High(Turns should alternate after every valid move).

Channel to actor: Internet connection (Wi-Fi, mobile data, or Ethernet).

Secondary actors: Player

Channel to secondary actors: Internet connection, screen, keyboard, mouse.

Open issues: What happens when a player never makes a move?

Use case: Update game board

Iteration: 1

Primary Actor: Game server

Goal in context: To ensure that a move made by one player is synchronized in real-time on the other player's device.

Preconditions:

1. Player logs into game platform.
2. The player is logged into the game.
3. The player is in an active game session.
4. It is the player's turn.

Trigger: Player makes a move

Scenario:

1. Player's move is validated and sent to the server.
2. The server updates the game board and synchronizes the move on the opponent's device in real-time.

Post conditions: The move made by a player on their game board is successfully reflected on the opponent's game board in real-time.

Exceptions: Player has a bad network connection.

Priority: High (A player should be able to see valid move made by the other player on their screen)

When available: 3rd iteration

Frequency of use: High (Game board is updated as soon as a valid move is made by a player)

Channel to actor: Internet connection (Wi-Fi, mobile data, or Ethernet).

Secondary actors: Player

Channel to secondary actors: Internet connection, screen, keyboard, mouse.

Open issues: N/A

Use Case: Fetch Game Board

Iteration: 1

Primary Actor: Player

Goal in Context: To ensure that the game board is properly loaded from the server and displayed correctly on the player's screen before the game starts.

Preconditions:

1. The player is logged into the game.
2. The player has joined an active game session.
3. The game server is accessible.

Trigger: The player starts or joins a game session, requiring the game board to be fetched from the server.

Scenario:

1. The player logs into the game platform.
2. The player selects a game mode or joins an active session.
3. The player requests the game board from the server.
4. The server retrieves the game board data and sends it to the player's screen.
5. The player confirms that the game board is loaded and begins gameplay.

Postconditions:

1. The game board is successfully loaded and displayed.
2. The game state is synchronized with the server.

Exceptions:

1. Server fails to respond or is unreachable.
2. Data corruption occurs while fetching the board.
3. The player loses internet connection before the board is loaded.

Priority: High

When Available: 2nd iteration

Frequency of Use: High

Channel to Actor: Screen, internet connection, game server.

Secondary Actors: Game Server

Channel to Secondary Actors: Internet connection (Wi-Fi, mobile data, or Ethernet).

Open Issues:

1. How should the game handle failures when fetching the game board?

Use Case: Attempt Reconnection

Iteration: 1

Primary Actor: Player

Goal in Context: To ensure a smooth gaming experience by detecting a lost connection, attempting to reconnect the player to the game server, and resynchronizing the game state.

Preconditions:

1. The player is logged into the game.
2. The player is in an active game session.
3. The game has an active internet connection at the start.

Trigger: The player's network connection is lost or becomes unstable.

Scenario:

1. The player's network connection is lost.
2. The game detects the disconnection and notifies the player.
3. The game automatically attempts to reconnect within a set time limit.
4. If reconnection is successful, the latest game state is retrieved from the server.
5. The game board is resynchronized for the player.
6. If the connection is not restored within the time limit, the other player is given a win.

Postconditions:

1. If reconnection is successful, the player resumes the game without losing progress.

2. If reconnection fails, the game follows predefined rules (waiting for a limited time, forfeiting the match).

Exceptions:

1. The server is down or unreachable.
2. The game session has expired due to inactivity.

Priority: High

When Available: 3rd iteration

Frequency of Use: Moderate(Depends on network stability and player connectivity).

Channel to Actor: Screen notifications, network status indicator.

Secondary Actors: Game Server

Channel to Secondary Actors: Internet connection (Wi-Fi, mobile data, or Ethernet).

Open Issues:

1. How long should the game wait before marking the player as disconnected?
2. Should the player be given manual reconnection options
3. What happens if the player reconnects, but the game session has already ended?

Use case: End game (win/lose/quit)

Iteration: 1

Primary Actor: Player

Goal in context: To ensure the game session is concluded properly when a player wins, loses, or quits, updating the game state accordingly while allowing a short window for reconnection in case of disconnection.

Preconditions:

1. The player is logged into the game.
2. The player is in an active game session.
3. The game session has reached a win/loss condition, or a player has disconnected or quit.

Trigger:

1. A player achieves a winning condition.
2. A player manually quits the game.
3. A player disconnects from the game.

Scenario:

1. A player logs into the game platform.
2. The player starts or joins an active game session.
3. The game progresses until a win/loss condition is met (e.g. checkmate in chess or four in a row in Connect Four).
4. The system detects the game ending condition.
5. If a player manually quits or disconnects, the game starts a reconnection timer (e.g. 30 seconds).
6. If the disconnected player reconnects within the time limit, they resume the game.
7. If the player does not reconnect in time, the remaining player is declared the winner.
8. The system prepares to send the game results to the server.

Post conditions:

1. The game is officially marked as finished.
2. The winner and loser are determined based on the final game state or disconnection timeout.
3. The match result is recorded and prepared for leaderboard updates.

Exceptions:

1. A server error prevents the match result from being updated.
2. A player disputes the match result due to a network issue.

Priority: High (Ensures a fair and smooth conclusion to each game session).

When available: 3rd iteration

Frequency of use: High (Occurs at the end of every game session).

Channel to actor: Screen, game interface

Secondary actors: Game server

Channel to secondary actors: Network connection

Open issues:

1. How long should the reconnection window be before awarding the win?
2. How should the leaderboard handle cases where players repeatedly quit?

Use case: Send game result to server

Iteration: 1

Primary Actor: Server

Goal in context: To ensure that the final game results are sent and stored on the server for leaderboard updates and game history tracking, accounting for cases where a player disconnected before the match ended.

Preconditions:

1. The game session has ended (win/loss/quit).
2. The server is available to receive game results.
3. The player is still connected to the network or has reconnected in time.
4. **Trigger:** The game reaches an end-state and results need to be recorded.

Scenario:

1. The game session concludes with a winner, loser, or quit action.
2. If a player disconnected, the system first checks whether they reconnected within the allowed time.
3. If the disconnected player did not reconnect, the remaining player is declared the winner.
4. The system compiles match details (players involved, final scores, time played, disconnection details, etc.).
5. The compiled data is formatted and sent to the game server.
6. The server receives the game result and stores it in the database.
7. The server updates player statistics, rankings, and leaderboards.
8. The updated information is made available for player profiles and public leaderboards.

Post conditions:

1. The game results are successfully stored in the database.
2. Player statistics are updated, including tracking disconnections if applicable.
3. The leaderboard is refreshed with the latest rankings.

Exceptions:

1. Network failure prevents the result from being sent.

2. The server is down or does not respond.
3. Data corruption occurs while transmitting match results.

Priority: High (Accurate game tracking is essential for leaderboard and player experience).

When available: 3rd iteration

Frequency of use: High (Occurs at the end of every game session).

Channel to actor: Server API, network connection

Secondary actors: Game server, leaderboard system

Channel to secondary actors: Internet connection

Open issues:

1. Should repeated disconnections affect a player's ranking or matchmaking score?
2. How should the system handle intentional quitting?

